When CPU Fault Actuation Does Not Manifest as an SLO Violation: An Evidence-Bounded Microservice Case Study

Anonymous Author(s)

Adviser-review status. Submission-layout candidate only. Receipt-level replay remains unavailable at 0/21 expected pairs. No model was trained or evaluated, and no runtime or external-validity claim is implied.

Abstract

Fault injection is often treated as evidence of a service failure once the injector reports success. This shortcut conflates physical actuation with an observable service-level outcome and can contaminate later early-warning or root-cause evaluations. We report an evidence-bounded case study of CPU stress against recommendationservice in a local Online Boutique deployment. The study covers two workload levels and three CPU-stress profiles. Across 35 unique attempts, fail-closed gates retained 21 valid runs (six normal and 15 fault) and preserved 14 invalid attempts outside scientific outcome denominators. The valid fault runs showed ordered descriptive CPU actuation: mean increases were 50.349, 99.649, and 146.941 millicores at 10 users, and 51.098, 93.987, and 140.435 millicores at 15 users. Nevertheless, none of the 15 valid fault runs manifested the prospectively frozen latency/error SLO, so the dataset contains no eligible positive lead-time event. We contribute a measured separation between actuation and manifestation, an auditable validity-denominator design, and a layered reproducibility account that distinguishes tracked evidence from unavailable receipt-level replay. The result is a bounded negative result for the tested host and profiles, not a universal claim about CPU faults or a predictive-model evaluation.

CCS Concepts: Software and its engineering to Software reliability; Computer systems organization to Cloud computing; General and reference to Empirical studies; Reproducibility.

Keywords: microservices, fault injection, negative results, SLO manifestation, reproducibility, evidence governance, chaos engineering

Evidence boundary: 35 attempts; 21 valid runs; 14 invalid attempts; 15 valid fault runs; 0/15 frozen-SLO manifestations; receipt replay 0/21.

1. Introduction

Microservice dependability studies commonly use fault injection to create controlled incidents for anomaly detection, early warning, or root-cause analysis. A successful injector command, however, establishes only that an operation was requested or executed. It does not establish that the target resource changed by the intended magnitude, that users experienced a service-level effect, or that the resulting run is eligible for scientific analysis. Treating these stages as interchangeable can create false positives, inflate sample sizes, and encourage model claims unsupported by event-level ground truth.

Prior work has already established multimodal and graph-based microservice RCA. MULAN combines metrics and logs with causal-graph learning and ranking [1], while GALR combines metrics, logs, traces, graph attention, LLM-derived priors, and recovery planning [2]. Consequently, this study does not claim novelty from combining telemetry, graphs, or LLMs. Nor does it present a predictive model. Instead, it examines an earlier experimental question: when CPU fault profiles produce measurable physical actuation, do they also produce the frozen service-level outcome needed to define positive early-warning incidents?

We ask three research questions:

- RQ1 - Physical actuation: Did the tested low, medium, and high CPU-stress profiles produce descriptively repeatable actuation under the recorded 10-user and 15-user workloads?
- RQ2 - Frozen-SLO outcome: Did any of the 15 valid fault runs manifest the frozen SLO or yield an eligible positive lead-time event?
- RQ3 - Evidence governance: Did fail-closed gates preserve an auditable separation between 21 valid runs and 14 invalid attempts?

The paper makes three bounded contributions:

- W5-C01: It empirically separates physical fault actuation from frozen-SLO manifestation in the tested same-host setting.
- W5-C02: It demonstrates how fail-closed validity accounting preserves all 35 attempts while restricting outcome denominators to eligible runs.
- W5-C03: It reports reproducibility in layers: tracked registry/report evidence is deterministically checkable, while receipt-level replay remains explicitly unavailable at zero of 21 expected pairs.

These contributions are methodological and descriptive. They are not a new RCA algorithm, an LLM/GAT architecture, a causal treatment-effect estimate, or an externally generalized conclusion.

2. Related Work

2.1 Multimodal and graph-based RCA

MULAN learns modality-specific and modality-invariant representations from metrics and logs, fuses causal graphs using KPI-aware attention, and ranks root causes through random walk with restart [1]. Its three evaluation datasets contain 4, 5, and 5 recorded system faults. GALR extends the design space with trace-derived topology, graph attention, LLM semantic priors, and retrieval-augmented recovery planning [2]. GALR evaluates post-event localization and offline action-level recovery quality; it reports injection-time-aligned window labels, two-run averages, and no statistical significance test. These systems solve tasks different from the present study. They establish that "multimodal RCA" and "LLM plus GAT" are prior-art design patterns, while our work evaluates whether valid positive incidents exist before such model comparisons are attempted.

2.2 LLM agents and operational safety

Tool-augmented LLM agents can assist diagnosis but can also apply shallow fixes, misinterpret stale signals, misuse tools, and take unnecessary or potentially harmful actions [3]. The broader LLM4AIOps literature spans detection, diagnosis, RCA, and remediation [5]. We do not evaluate an LLM. These findings instead motivate the project's governance boundary: analysis artifacts cannot authorize autonomous mutation of a running system, and any future LLM verdict must remain tied to timestamped evidence.

2.3 Proactive SLO evaluation

Proactive SLO work distinguishes violation prediction, tail-latency prediction, prefix early warning, and actionable outputs [4]. Early-warning evaluation should combine a precision constraint with earliness, coverage, and false-alarm measures, while explicitly distinguishing proxy labels from real SLO semantics. Our study cannot compute those metrics: the 15 valid fault runs contain zero positive manifestations. Windows within a run are not independent incidents and cannot manufacture positive sample size.

3. Study Design

3.1 System and scope

The case study uses a local Online Boutique deployment and targets recommendationservice. All analyzed runs were recorded on the same host. Two versioned workloads were used: 10 virtual users and 15 virtual users, each with one worker. Three CPU-stress severities - low, medium, and high - were evaluated at both workloads. The terms "low," "medium," and "high" identify versioned fault profiles; the physical-effect results are derived from measured steady-minus-baseline CPU differences rather than from labels alone.

The population is therefore limited to the recorded host, deployment, workloads, target service, fault profiles, and SLO. The study does not support generalization to production clusters, different services, other resource limits, or other fault classes.

3.2 Treatment and workload contract

The target was the recommendationservice server container with a 200 mCPU limit. Versioned low, medium and high profiles requested approximately +50, +100 and +150 mCPU through an in-container Python duty-cycle worker operating on a 100 ms cycle. Each profile used a 120 s linear ramp and 300 s steady phase. Automatic termination was mandatory, total worker time was bounded by 450 s, and worker started/completed events supplied the canonical lifecycle UTC boundary with a +/-5 s duration tolerance.

The 15-user profiles changed only the workload binding and profile identity; fault physics, target, durations, effect gates and SLO remained equivalent to their 10-user counterparts.

Both workload profiles used one loadgenerator replica, one user spawned per second, a uniform 1-10 s wait, random seed 1, Locust 2.43.0 and the same pinned task mix: index 1, setCurrency 2, browseProduct 10, addToCart 2, viewCart 3 and checkout 1. The only planned workload difference was 10 versus 15 concurrent users. Each run used a 300 s warm-up and 300 s normal baseline. The full fault lifecycle was 300/300/120/300/300 s for warm-up, baseline, ramp, steady and cooldown.

3.3 Physical-effect eligibility

Prometheus target-container CPU measurements independently tested actuation. Baseline and steady phases each required at least 48 adjacent real-scrape CPU rate intervals out of 60 expected intervals at the observed five-second cadence. Low, medium and high profiles additionally required mean steady-minus-baseline increases of at least 25, 50 and 75 mCPU, respectively. Both coverage and magnitude had to pass. Injector command success alone was insufficient. Throttling was reported but did not count as SLO manifestation.

3.4 Frozen manifestation rule

The prospectively frozen SLO uses five-second windows anchored to the normal baseline start, with no phase realignment. Manifestation requires either product-service p95 latency above 345.992 ms for three consecutive populated windows or a global error rate greater than zero for three consecutive populated windows. Empty windows reset the streak. The threshold and consecutive-window rule were not selected after viewing fault outcomes.

3.5 Run eligibility and accounting

The registry contains 35 unique attempts. Twenty-one are valid: six normal controls and 15 fault runs. Fourteen attempts are invalid. A run enters a scientific outcome denominator only when its canonical registry status and tracked evidence satisfy the applicable physical-effect, lifecycle, host, identity, and manifestation contracts. Invalid attempts are preserved as operational provenance but are not relabeled, deleted, retried under the same identifier, or counted as successful/negative treatments.

This fail-closed rule prevents operational attempts from being confused with independent scientific observations. The validity proportion, 21/35, is an accounting fact rather than a treatment success rate.

3.6 Analysis

For each workload-by-severity cell, we report the arithmetic mean, sample standard deviation, coefficient of variation, and range of measured CPU increase across valid runs. The coefficient of variation is sample SD divided by the mean. Sample SD is not a confidence interval. Cell sizes are two or three runs, so no population inference or statistical-significance claim is made. Warning lead time is eligible only when a valid fault run manifests the frozen SLO.

4. Results

4.1 Attempt accounting

The 35 attempts comprise 21 valid runs and 14 invalid attempts. The valid set contains six normal controls and 15 fault runs. All invalid attempts remain in the registry, but none contributes to the physical-effect or manifestation denominators.

The 35 unique attempts contain 21 valid runs: six normal controls and 15 fault runs. Fourteen attempts remain invalid and visible. For audit explanation only, they group into two invalid normal baselines, five fault attempts that stopped before actuation and seven attempts with later lifecycle, physical-effect or finalization failures. These groups are operational explanations rather than new scientific labels. None of the 14 attempts enters an effect or manifestation denominator.

4.2 Physical actuation

Workload	Severity	n	Mean increase (mCPU)	Sample SD	CV
10 users	Low	3	50.349	1.801	3.576 %
10 users	Medium	3	99.649	4.930	4.947 %
10 users	High	3	146.941	3.313	2.254 %
15 users	Low	2	51.098	2.751	5.384 %
15 users	Medium	2	93.987	0.661	0.704 %
15 users	High	2	140.435	7.460	5.312 %

Table 1. Descriptive physical actuation by workload and severity.

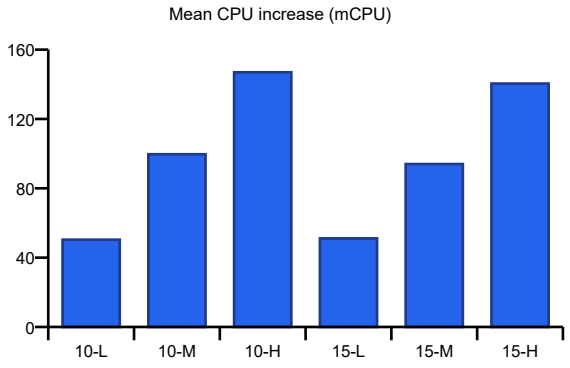


Figure 1. Mean measured CPU increase. Labels combine workload and severity; bars are descriptive means, not causal estimates.

Observed steady-minus-baseline CPU-increase ranges were:

- 10 users: low 48.463-52.050, medium 93.994-103.042, high 143.819-150.416 mCPU;
- 15 users: low 49.153-53.044, medium 93.519-94.454, high 135.160-145.710 mCPU.

The reported SD values are sample standard deviations. Ranges, SD and CV describe accepted runs only; none is a confidence interval or causal estimate.

The ordered means support the bounded conclusion that the versioned profiles produced distinguishable physical actuation in the recorded runs. They do not establish an SLO effect or external generality.

4.3 Manifestation and lead time

None of the 15 valid fault runs manifested the frozen SLO: 0/15. Consequently, no run had an eligible positive warning interval. This is zero positive lead-time evidence, not evidence that an early-warning method succeeded or failed. An SLO event observed in an invalid normal attempt remains excluded under the validity rule and cannot be reassigned to a valid denominator.

4.4 Answering the research questions

RQ1 is supported descriptively for the tested cells: measured CPU actuation was ordered across low, medium, and high profiles at both workloads. RQ2 is answered negatively within scope: the valid fault set produced no frozen-SLO manifestation and no eligible positive lead-time event. RQ3 is supported at the tracked-evidence layer: the registry, reports, and verifier preserve the 21-valid/14-invalid separation. RQ3 remains incomplete at the stronger receipt layer because zero of 21 expected receipt/SHA-manifest pairs are available.

5. Artifact and Reproducibility Design

The artifact distinguishes four layers. First, canonical governance documents define protocol, labels, results, and claim boundaries. Second, tracked run reports and JSON contracts support the 35-attempt accounting and the 15 valid fault outcomes. Third, deterministic scripts regenerate tables, the figure, and bounded manuscript checks. Fourth, final receipt bundles would independently replay archive provenance, but this layer is unavailable at 0/21.

The Week 3 derivation is pinned to source revision `0ecb6011189c0a47b46019ec1d626ae7043e19e0`. Generated text uses LF line endings, and exact comparisons target Git blob bytes rather than platform-converted checkout bytes. This rule corrects a portability defect in the initial pre-commit manifest without changing any scientific value. The complete repository-only replay requires Git, Python's standard library, and PowerShell; it does not require Kubernetes, Docker, Prometheus, Locust, a GPU, network access, or an LLM API.

6. Discussion

6.1 Actuation is a prerequisite, not an outcome

The results show why an injector exit code is insufficient for incident construction. Physical actuation was measurable and ordered, yet the frozen service-level outcome did not appear. A downstream model trained by labeling all injected windows as positive would therefore learn injection scheduling or resource movement rather than manifested service failure.

6.2 Negative results constrain future model claims

The absence of positive manifestations means the current CPU evidence cannot support event-level rule, logistic, boosted-tree, LLM, or graph-model performance. It also cannot estimate warning coverage, false-alarm trade-offs, or lead-time distributions. Future confirmatory work remains governed by a target of 60 independent positive incidents plus 60 normal control runs. Windows within one incident cannot substitute for independent runs.

6.3 Invalid attempts remain informative but non-scientific

Preserving 14 invalid attempts reveals operational failure modes and prevents selective disappearance of inconvenient runs. Their presence can guide tooling improvements, but they cannot support effect estimates, manifestation rates, or robustness claims. This distinction is especially important in complex microservice experiments, where infrastructure failures can outnumber eligible scientific observations.

7. Threats to Validity

7.1 Construct validity

CPU actuation and service degradation are different constructs. The frozen SLO reduces post-hoc flexibility but may not capture every user-relevant degradation. Conversely, an injection label alone is not a valid proxy for an SLO event.

7.2 Internal validity

The same-host design reduces some environmental variation but exposes results to host scheduling and shared-resource interference. Fail-closed gates reduce outcome-dependent inclusion, yet cell sizes remain small and residual run-to-run variation cannot be separated from treatment effects.

7.3 Conclusion validity

The analysis is descriptive. Sample SD and CV characterize the observed runs; they are not confidence intervals. The 0/15 result does not prove that CPU faults never cause service degradation or that predictive methods are ineffective in other conditions.

7.4 External validity

Evidence comes from one host, one local deployment, one target service, two workloads, and three CPU profiles. No production, cross-host, cross-service, or cross-fault generalization is supported.

7.5 Reproducibility validity

Tracked evidence and derived outputs are deterministically checkable, but receipt-level replay is unavailable. A passing verifier establishes consistency with tracked sources; it cannot reconstruct absent artifacts or prove exact reconstruction of the original runtime environment.

8. Ethics and Responsible Operation

The study uses a synthetic benchmark application and controlled local fault injection; it contains no human-subject data or personal user telemetry. The publication artifact does not authorize deployment changes, autonomous LLM actions, or new fault execution. Generated recovery or diagnosis commands are outside the study scope. Invalid runs and negative outcomes are retained to reduce selective reporting.

9. Conclusion

In this bounded microservice case study, CPU-stress profiles produced ordered physical actuation but no frozen-SLO manifestation across 15 valid fault runs. The result demonstrates why fault actuation, service manifestation, validity accounting, and artifact replay must be evaluated separately. The evidence can support a reproducible tracked-layer negative result and methodological account, but not a predictive-model, causal-effect, or external-validity claim. Future work requires independently sampled positive incidents, normal controls, frozen feature windows, and paired model-versus-rule evaluation before early-warning performance can be reported.

Artifact Availability Statement

An anonymized repository artifact is planned for adviser and venue review. The current candidate package contains deterministic tracked-evidence replay, manuscript checks, and an explicit 0/21 receipt limitation. The history-free archive candidate excludes Git history, canonical governance files, raw tracked evidence, and absent receipt bundles. It is not yet a public release, DOI deposit, or submission artifact.

References

[1] L. Zheng, Z. Chen, J. He, and H. Chen. 2024. MULAN: Multi-modal Causal Structure Learning and Root Cause Analysis for Microservice Systems. The Web Conference. DOI: 10.1145/3589334.3645442.

[2] W. Zhang, Z. Yang, F. Peng, L. Zhang, Y. Chen, and R. Chen. 2026. GALR: Graph-Based Root Cause Localization and LLM-Assisted Recovery for Microservice Systems. Electronics 15(1), 243. DOI: 10.3390/electronics15010243.

[3] A. Cornacchia et al. 2025. Between Promise and Pain: The Reality of Automating Failure Analysis in Microservices with LLMs. APSys. DOI: 10.1145/3725783.3764388.

[4] M. Yu et al. 2026. From Distributed Tracing to Proactive SLO Management: A Mini-Review of Trace-Driven Performance Prediction for Cloud-Native Microservices. Frontiers in Computer Science 8. DOI: 10.3389/fcomp.2026.1783945.

[5] L. Zhang et al. 2025. A Survey of AIOps in the Era of Large Language Models. ACM Computing Surveys 58(2). DOI: 10.1145/3746635.